ApexPay

A digital wallet and payments API

Project documentation — architecture, data model, API reference, security and deployment

---

Application: ApexPay — wallet, transfers, merchant checkout and refunds

Backend: ASP.NET Core (.NET 10) REST API + PostgreSQL

Frontend: React 19 single-page web app

Live API: https://apexpay-zqfr.onrender.com

Live web app: https://apexpay-8ty.pages.dev

Source: https://github.com/ErA-16/ApexPay

Document date: October 2026

Contents

1. Overview
2. Feature summary
3. Technology stack
4. Architecture
5. Domain and data model
6. How money moves (double-entry ledger)
7. Idempotency
8. Concurrency and locking
9. API reference
10. Error handling and status codes
11. Security
12. Frontend
13. Configuration and environment
14. Database migrations
15. Testing
16. Deployment
17. Known limitations and roadmap
18. Appendix: repository layout and glossary

1. Overview

ApexPay is a small, production-style digital wallet and payments system. It lets a person open an account, hold money in one or more wallets, move money between wallets, pay a merchant, and receive refunds. Every movement of money is recorded in an append-only ledger so the balance of any wallet can be proved from its history.

The project is deliberately built to look and behave like a real payments backend rather than a toy CRUD example. The interesting parts are not "create a row and read it back" but the things that make money systems hard: **never losing or duplicating money**, and **staying correct when two people act at the same time**. That is where most of the design effort sits.

What problem it solves

A wallet system has to answer a handful of hard questions safely:

- How do you stop a retried request (a flaky network, a double-click) from charging someone twice?
- How do you move money between two accounts so it can never be lost or created out of thin air?
- What happens when two transfers touch the same wallets at the same instant?
- How do you prove a wallet's balance is correct?
- How do you refund a payment when the original wallet no longer exists?

ApexPay answers all five in the design: idempotency keys, a double-entry ledger, atomic conditional updates with a fixed lock order, database-level constraints as a last line of defence, and a refund path that can open a fresh wallet to receive the money.

Scope of this document

This document describes the whole project: what it does, how it is structured, the data it stores, every API endpoint, the security model, how it is tested, and how it is deployed. It is written to be read by a developer or a reviewer who wants to understand the system without reading every source file.

2. Feature summary

Feature	What it does
Accounts & auth	Email + password sign-up and login. Passwords are hashed with BCrypt (SHA-384). Login issues a signed JWT access token.
Wallets	Each account can hold multiple named wallets. A wallet has a 10-digit account number, a currency, a balance and a status (ACTIVE / CLOSED). Wallets can be renamed and closed.
Funding	Adds money to a wallet and writes a Funding transaction and a ledger credit.
Transfer	Moves money from one wallet to another. Writes a single transaction with two ledger entries (one debit, one credit).
Checkout	A user pays a merchant wallet (a special wallet whose role is MERCHANT). Records the order id.
Refund	Reverses all or part of a checkout. If the original wallet is closed, the money is placed in a dedicated "Refund" wallet for that account.
Ledger & history	Every wallet has a paginated history of debit and credit entries, plus a consistency check comparing the ledger total to the stored balance.
Idempotency	Money-moving endpoints require an `Idempotency-Key` header so a retried request returns the original result instead of repeating the operation.
Rate limiting	Sign-up and login are rate limited per client IP to slow down abuse.
Web app	A React single-page app covering login, sign-up, dashboard, wallets, funding, sending, paying and refunding.

3. Technology stack

Layer	Technology	Notes
Runtime / SDK	.NET 10 (SDK 10.0.401)	Target framework `net10.0`
Web framework	ASP.NET Core controllers	Attribute-routed `[ApiController]` controllers
ORM	Entity Framework Core 10.0.12	Code-first with migrations
Database provider	Npgsql.EntityFrameworkCore.PostgreSQL 10.0.3	PostgreSQL (hosted on Neon)
Naming convention	EFCore.NamingConventions 10.0.1	Global snake_case table/column names
Auth	Microsoft.AspNetCore.Authentication.JwtBearer 10.0.13	JWT bearer tokens
Password hashing	BCrypt.Net-Next 4.2.0	`EnhancedHashPassword` / `EnhancedVerify`, SHA-384
API docs	Microsoft.AspNetCore.OpenApi 10.0.12	OpenAPI document in Development
Frontend	React 19 + Vite 7	TypeScript 5.9, react-router-dom 7
Hosting	Render (API, Docker) + Cloudflare Pages (web)	PostgreSQL on Neon

4. Architecture

ApexPay is a two-project solution (`ApexPay.slnx`): the `ApexPay` web API and the `ApexPay.Test` integration test project. The web API is layered so that HTTP concerns and business rules stay separate.

Layers

Layer	Responsibility
Controllers	HTTP only: routing, model binding, reading the caller's user id from the JWT, light validation, and translating exceptions into status codes.
Services	Business logic and the unit of work: `WalletService` (money movement), `AccountService` (accounts and tokens), `IdempotencyService` (retry safety).
Data layer	`ApexPayDbContext` maps the entities to PostgreSQL and holds all the constraints and indexes.
Database	PostgreSQL enforces the invariants (unique keys, check constraints, foreign keys) so the rules hold even if application code has a bug.

> **Guiding rule:** the controller decides *what* is allowed at the edge, the service decides *how* money moves, and the database is the last line of defence that refuses anything that would break an invariant.

Request flow (a money movement)

1. The client sends a request to the API with a valid JWT and an `Idempotency-Key` header.
2. The controller authenticates the caller, extracts the user id from the token's `sub` claim, and does edge validation (for example: an amount must have at most two decimal places).
3. The controller calls the idempotency orchestrator. If this key was already used for this path, the stored response is replayed and no work happens.
4. Otherwise the request is claimed as "in progress" and handed to `WalletService`.
5. The service opens a database transaction, performs atomic conditional updates, writes the transaction and ledger rows, marks the idempotency key completed, and commits.
6. The controller returns the result, or the matching status code if a business rule was violated.

Project structure

```
ApexPay/                 Web API
  Controllers/           AccountController, WalletController
  Service/               WalletService, AccountService, IdempotencyService
  Service/Exceptions/    WalletException, AuthException and their subclasses
  Dto/                   Request and response models (never entities)
  Models/                EF entities and enums
  Database/              ApexPayDbContext, DatabaseSeeder
  Migrations/            EF Core migrations
ApexPay.Test/            xUnit integration tests
ApexPay.Web/             React single-page app
Dockerfile               API container image
```

5. Domain and data model

Five tables hold all state: accounts, wallets, transactions, ledger entries and idempotency records. All names are snake_case (applied globally by the naming convention) and all primary keys are UUIDs generated in the application.

Entities at a glance

Entity	Purpose
Account	A user. Holds full name, unique email and an optional password hash.
Wallet	A balance container owned by an account. Has an account number, name, currency, balance, status and role.
Transaction	A single movement of money (funding, transfer, payment or refund). Has a unique reference.
LedgerEntry	One debit or credit line belonging to a transaction and a wallet. The ledger is append-only.
Idempotency	Remembers the result of a money-moving request so retries are safe.

accounts

Column	Type	Rules
`id`	uuid	Primary key
`full_name`	varchar(100)	Not null
`email`	varchar(255)	Not null, unique (`uq_account_email`)
`password_hash`	text	Nullable (the seeded merchant has none)
`created_at`	timestamptz	UTC

wallets

Column	Type	Rules
`id`	uuid	Primary key
`account_number`	varchar(10)	Not null, unique (`uq_wallet_account_number`)
`name`	varchar(50)	Not null
`account_id`	uuid	Not null, FK to accounts (restrict delete)
`currency`	varchar(3)	Default `NGN`
`balance`	numeric(18,2)	Not null, default 0. Check `balance >= 0`
`status`	varchar(20)	ACTIVE / CLOSED, default ACTIVE
`role`	varchar(20)	USER / MERCHANT, default USER
`created_at`	timestamptz	UTC

Indexes: `uq_wallet_account_id_name` is unique on (account_id, name) but only for `status <> 'CLOSED'`, so a closed wallet's name can be reused. `ix_wallet_account_id_created_at` supports listing a user's wallets in a stable order. `chk_positive_balance` stops a balance from ever going negative.

transactions

Column	Type	Rules
`id`	uuid	Primary key
`reference`	varchar(64)	Not null, unique (`uq_transaction_reference`)
`type`	varchar(20)	FUNDING / TRANSFER / PAYMENT / REFUND
`status`	varchar(20)	SUCCESSFUL / FAILED
`amount`	numeric(18,2)	Not null, check `amount > 0`
`source_wallet_id`	uuid	Nullable (funding has no source), FK restrict
`destination_wallet_id`	uuid	Not null, FK restrict
`parent_transaction_id`	uuid	Nullable, FK restrict (links a refund to its original payment)
`order_id`	varchar(64)	Nullable (checkout)
`narration`	varchar(200)	Nullable
`failure_reason`	text	Nullable
`created_at`	timestamptz	UTC

Index `ix_transaction_parent` supports summing refunds against a parent payment.

ledger_entries

Column	Type	Rules
`id`	uuid	Primary key
`transaction_id`	uuid	Not null, FK restrict
`wallet_id`	uuid	Not null, FK restrict
`entry_type`	varchar(10)	DEBIT / CREDIT
`amount`	numeric(18,2)	Not null, check `amount > 0`
`created_at`	timestamptz	UTC

Index `ix_ledger_wallet_created` supports reading a wallet's history in time order.

idempotency_records

Column	Type	Rules
`key`	varchar(128)	Primary key (the client-supplied GUID as text)
`request_path`	varchar(255)	Not null (which endpoint the key was used on)
`response_code`	integer	Nullable, filled when finished
`response_body`	text	Nullable, the stored response
`status`	varchar(20)	INPROGRESS / COMPLETED / FAILED
`created_at`	timestamptz	UTC (used to detect stuck keys)

Enumerations

Enum	Values
WalletStatus	Active, Closed
WalletRole	User, Merchant
TransactionType	Funding, Transfer, Payment, Refund
TransactionStatus	Successful, Failed
LedgerEntryType	Debit, Credit
IdempotencyStatus	InProgress, Completed, Failed

Enums are stored as uppercase text, which keeps the database readable and stable if enum values are reordered in code.

Money

All money is stored as `numeric(18,2)` — two decimal places, never floating point. Amounts are validated at the edge to have at most two decimal places and to fall within the allowed range before any work begins.

6. How money moves (double-entry ledger)

Every change to a wallet balance is paired with a ledger entry, and every transaction that touches two wallets writes two entries: one debit and one credit of the same amount. Money is never created or destroyed by a transfer — it only changes hands.

Operation	Transaction type	Ledger entries written
Fund	FUNDING	One credit to the wallet
Transfer	TRANSFER	Debit the source, credit the destination
Checkout	PAYMENT	Debit the user wallet, credit the merchant wallet
Refund	REFUND	Debit the merchant wallet, credit the buyer (or the "Refund" wallet)

Proving a balance

A wallet's stored balance should always equal the sum of its ledger credits minus its ledger debits. The history endpoint returns both figures and a boolean:

- `ledgerCalculatedBalance` — credits minus debits from the ledger.
- `walletStoredBalance` — the balance column on the wallet.
- `isConsistent` — true when the two agree.

This is a cheap, always-on self-check: if the two ever disagree, something wrote a balance without a matching ledger entry, and it is visible immediately without a separate audit job.

Refunds when the original wallet is gone

A buyer may have closed the wallet they paid from. ApexPay does not lose the refund: it looks for a wallet named "Refund" on that account, and if none exists it opens one and credits the money there. The refund response includes a `createdWallet` flag so the client knows a new wallet appeared. Partial refunds are supported: each refund records the original payment as its parent, and the amount already refunded is summed to work out what remains.

7. Idempotency

Networks retry. Users double-click. A payments API must treat the same request arriving twice as *one* operation. ApexPay does this with an `Idempotency-Key` header.

The rule

`fund`, `transfer`, `checkout` and `refund` require an `Idempotency-Key` header whose value is a GUID. The key is scoped to the request path, so the same key can be reused on a different endpoint.

The lifecycle of a key

1. **Replay.** If a finished record exists for this path and key, its stored response (status code and body) is returned unchanged. No work is done.
2. **Claim.** Otherwise the request inserts a record with status INPROGRESS. A unique-key collision means someone else is already handling this key.
3. **Race.** If the claim fails, the request waits up to 3 seconds (polling every 150 ms) for the other caller to finish. If a result appears, it is replayed; if not, the caller gets a 409 Conflict instead of running the work twice.

4. **Complete.** On success the service writes the response into the record and marks it COMPLETED, inside the same transaction as the money movement.
5. **Fail.** If a business rule throws, the record is marked FAILED with the matching status code and message, so the client sees the same error on a retry instead of a different one.

Stuck keys

If a process dies mid-request, a key can be left INPROGRESS forever. A record older than the stale threshold (`Idempotency:StaleAfterMinutes`, default 2) can be taken over by exactly one caller, using an atomic conditional update so only one caller wins. This prevents a single crash from permanently blocking a key.

> **Why validate before claiming:** amount format and range are checked in the controller *before* the key is claimed, so a malformed request never burns a working key. Every exit after the claim records either a success or a failure.

8. Concurrency and locking

Two requests can touch the same wallets at the same time. ApexPay keeps this safe without long row locks on the hot path, using the database to make each step atomic.

Atomic conditional updates

Balances are changed with a single `ExecuteUpdateAsync` statement that carries every condition in its `WHERE` clause. A debit, for example, only succeeds if the wallet exists, belongs to the caller, is active, and has enough balance:

```
UPDATE wallets
SET balance = balance - @amount
WHERE id = @walletId
  AND account_id = @userId
  AND status = 'ACTIVE'
  AND balance >= @amount;
```

This makes "check the balance, then subtract" a single atomic operation. There is no gap between the check and the write for another request to slip into. The number of rows changed tells the code exactly what happened (0 rows means it was rejected; 1 means it succeeded).

Consistent lock order

When two wallets are involved (transfer, checkout, refund), the two updates are always applied in the same order everywhere: by comparing the wallet ids. Because every code path orders the two wallets the same way, two opposite transfers between the same pair of wallets cannot deadlock each other.

Reference and account-number collisions

Account numbers and transaction references are generated randomly and are unique. On the rare collision, the database refuses the duplicate and the service retries with a new value (up to five times).

The errors are caught by their specific constraint name, never by a blanket catch.

Refund locking

Refunds are the one place a lock is held on purpose: the original transaction row is selected `FOR UPDATE`, and, when a refund wallet must be created, the owning account row is locked too. This serialises concurrent refunds of the same payment so the "already refunded" total cannot be raced.

> **Defence in depth:** application code does the right thing, and the database refuses to store anything that breaks a rule. Balances cannot go negative (`balance >= 0`), amounts must be positive, emails, account numbers, references and (account, name) pairs are unique. If the code has a bug, the constraint catches it rather than corrupting data.

9. API reference

All routes are under `/api`. Authenticated routes expect `Authorization: Bearer <token>`. Money-moving routes also require an `Idempotency-Key` header. Bodies and responses are JSON.

Account endpoints

Method	Route	Auth / notes
POST	`/api/account/signup`	Public, rate limited. Creates an account.
POST	`/api/account/login`	Public, rate limited. Returns a JWT.
GET	`/api/account/me`	Bearer. Returns the current user's profile.

POST /api/account/signup

Request body (validation in brackets):

```
{
  "firstName": "Ada",          // required, max 45
  "lastName": "Obi",           // required, max 45
  "email": "ada@example.com",  // required, valid email, max 255
  "password": "Passw0rd123"    // required, 8-100
}
```

```
curl -X POST https://apexpay-zqfr.onrender.com/api/account/signup \
  -H "Content-Type: application/json" \
  -d "{\"firstName\":\"Ada\",\"lastName\":\"Obi\",\"email\":\"ada@example.com\",\"password\":\'
```

Returns `201` with `{ id, fullName, email }`. A duplicate email returns `409`; invalid input returns `400`.

POST /api/account/login

```
{ "email": "ada@example.com", "password": "Passw0rd123" }
```

Returns `200` with `{ accessToken, expiresAt }`. Wrong password and unknown email both return `401` with the same generic message, so an attacker cannot tell whether an email is registered.

GET /api/account/me

Returns `200` with `{ id, fullName, email }`. Used by the web app's dashboard greeting.

Wallet endpoints

All wallet routes require a bearer token. The path prefix is `/api/wallet`.

Method	Route	Purpose / notable status codes
POST	`/api/wallet/create-wallet`	Create a wallet. 201; 409 duplicate name; 404 no account.
GET	`/api/wallet/wallets?page=1&limit=20`	List the caller's wallets, paginated. 200; 400 bad paging.
GET	`/api/wallet/{walletId}`	One wallet. 200; 404 (also when it belongs to someone else).
GET	`/api/wallet/{walletId}/history?page=1&limit=20`	Ledger history + consistency check. 200; 404; 400.
GET	`/api/wallet/by-account-number/{accountNumber}`	Look up an active wallet by account number (for sending money). 200; 404.
GET	`/api/wallet/merchant`	The settlement merchant wallet. 200; 404.
PATCH	`/api/wallet/{walletId}/close-wallet`	Close a wallet. 204; 409 balance not zero; 404.
PATCH	`/api/wallet/{walletId}/rename`	Rename a wallet. 204; 409 name taken; 404.
POST	`/api/wallet/fund`	Add money. Needs Idempotency-Key. 200; 400; 404; 409; 500.
POST	`/api/wallet/{walletId}/transfer`	Send money. Needs Idempotency-Key. 200; 400; 404; 409; 500.
POST	`/api/wallet/checkout`	Pay a merchant. Needs Idempotency-Key. 200; 400; 404; 409; 500.
POST	`/api/wallet/refund`	Refund a payment. Needs Idempotency-Key. 200; 400; 404; 409; 500.

POST /api/wallet/fund

```
Idempotency-Key: 3f1c1e8e-2f3a-4c0b-9a7e-1b2c3d4e5f60

{
  "walletId": "00000000-0000-0000-0000-000000000000",  // required
  "amount": 5000,                                        // 100 - 10,000,000
  "narration": "Top up"                                  // optional, max 200
}
```

Returns `200` with the transaction id, reference, new balance and the account details.

POST /api/wallet/{walletId}/transfer

```
{
  "destinationWalletId": "00000000-0000-0000-0000-000000000000",  // required
  "amount": 2500,                                                  // 100 - 10,000,000
  "narration": "Rent"                                              // optional
}
```

POST /api/wallet/checkout

```
{
  "userWalletId": "00000000-0000-0000-0000-000000000000",      // required
  "merchantWalletId": "00000000-0000-0000-0000-000000000000",  // required
  "amount": 1200,                                              // 100 - 10,000,000
  "orderId": "ORD-1234"                                        // required, max 64
}
```

POST /api/wallet/refund

```
{
  "originalTransactionReference": "TXN-CHK-20261016-ABCDEF",  // required
  "reason": "Customer changed mind"                           // optional, max 200
}
```

10. Error handling and status codes

Business rules are expressed as small, named exception classes. Each carries its own status code and message. The controller catches the type and returns the matching HTTP status, so the mapping is written once and is easy to audit. The mapping is pinned by a test, which treats it as a contract.

Exception	Status	Meaning
EmailExistException	409	Sign-up email already registered
InvalidCredentialsException	401	Wrong email or password
AccountNotFoundException	404	Referenced account does not exist
WalletNotFoundException	404	Wallet does not exist or is not active
WalletNotOwnedException	404	Wallet exists but belongs to someone else (deliberately 404, not 403, so ids are not confirmed)
WalletNameConflictException	409	Another open wallet already uses that name
WalletAlreadyClosedException	409	The wallet is closed
InsufficientBalanceException	409	Not enough money for the operation
WalletConflictException	409	Generic business conflict (e.g. non-zero balance on close, already refunded)
WalletBadRequestException	400	Request is not valid for the operation (e.g. not refundable)
WalletCreationFailedException	500	Could not generate a unique account number
WalletProcessingFailedException	500	Could not process after retries

> **Unexpected errors:** anything the client did not cause is logged with full context and returned as a safe `500` message. Internal details and stack traces are never sent to the client. Model-validation failures return the framework's standard ProblemDetails shape with a `400`.

11. Security

Area	How it is handled
Authentication	JWT bearer tokens signed with a symmetric key (HMAC-SHA256). The user id travels in the `sub` claim; issuer, audience and lifetime are all validated.
Password storage	BCrypt with SHA-384. Passwords are never stored or logged in plain text.
Login enumeration	Unknown email and wrong password return the same generic 401, so the API does not reveal which emails exist.
Ownership checks	Every wallet read and write is scoped to the token's user id. A wallet that belongs to someone else returns 404, never the data.
Input validation	Validated at the edge with data annotations (required, length, range, email) and explicit checks (decimal places, same-wallet guards). Bad input is rejected before any work.
Rate limiting	Sign-up and login are limited to 10 requests per 60 seconds per client IP; over the limit returns 429.
CORS	A single allowed origin (`Frontend:Origin`) is configured; not a wildcard.
Client IP behind a proxy	Forwarded headers are trusted one hop deep (`ForwardLimit = 1`) so the rate limiter keys on the real client IP even behind a host proxy, without letting a client spoof it.
Secrets	Connection strings, the JWT key and other secrets live in user secrets in development and environment variables in production. None are committed to the repository.
SQL safety	All queries are parameterised (EF Core or parameterised raw SQL). No query is built from strings.

12. Frontend

The web app is a React 19 single-page application built with Vite and TypeScript. Pages are loaded lazily and the production bundle is code-split, so the first load only ships what the first screen needs.

Pages and routes

Route	Screen
`/login`	Sign in
`/signup`	Create an account
`/`	Home dashboard: total balance, quick actions, wallet list
`/wallets`	All wallets, create a wallet
`/wallets/:id`	Wallet detail: balance, history, rename, close
`/fund`	Add money to a wallet
`/transfer`	Send money (by account number)
`/checkout`	Pay a merchant
`/refund`	Refund a payment

Every route except login and signup is wrapped in a `ProtectedRoute` that redirects to `/login` when there is no session.

Session and idle logout

The JWT is kept in `sessionStorage` (cleared when the tab closes). An idle watcher logs the user out after 30 minutes of inactivity and shows a one-minute warning first. A 401 from the API also clears the session.

API client and errors

One small client adds the bearer token and the idempotency key to each request, clears the session on 401, and turns error responses into friendly messages (including parsing the framework's ProblemDetails shape). Idempotency keys are generated with `crypto.randomUUID()`.

Money handling in the UI

Amounts are validated in the browser with the same rules as the API (at most two decimals, between 100 and 10,000,000) and formatted as `NGN 1,234.56`. Nothing is sent to the server until it passes.

Design

A small set of reusable components (headers, bottom navigation, status pills, confirm dialog, wallet selector, pagination) and a green identity (emerald `#0e7c57` to `#22c55e`) with self-hosted Inter and JetBrains Mono fonts. The layout is mobile-first with a bottom navigation bar on the main screens.

Screenshot placeholder: Login / Sign-up Drop a screenshot of the login or sign-up screen here.

Screenshot placeholder: Home dashboard Drop a screenshot of the dashboard (total balance, quick actions, wallets) here.

Screenshot placeholder: Wallets and wallet detail Drop screenshots of the wallet list and a wallet's history here.

Screenshot placeholder: Send / Pay / Refund Drop screenshots of the money-moving screens here.

13. Configuration and environment

Configuration is read from files in development and from environment variables in production. ASP.NET Core maps a double-underscore in an environment variable to a colon in configuration, so `Jwt__Key` sets `Jwt:Key`.

Backend settings

Key	Purpose
`ConnectionStrings:DefaultConnection`	PostgreSQL connection string. User secrets in development; environment variable in production.
`Jwt:Key`	Signing key for access tokens (required).
`Jwt:Issuer`, `Jwt:Audience`	Token issuer and audience.
`Jwt:AccessTokenMinutes`	Token lifetime, default 60 minutes.
`Idempotency:StaleAfterMinutes`	When an in-progress key may be taken over, default 2.
`RateLimit:Auth:PermitLimit`	Auth requests per window, default 10.
`RateLimit:Auth:WindowSeconds`	Rate-limit window in seconds, default 60.
`Frontend:Origin`	The single allowed CORS origin (required; validated at startup).
`AllowedHosts`	Host filtering.

At startup the API fails fast if `Jwt:Key` or `Frontend:Origin` is missing, or if `Frontend:Origin` is not a full URL with a scheme, no trailing slash and no path.

Frontend build variables

Variable	Purpose
`VITE_API_BASE_URL`	The API base URL, inlined at build time. The app refuses to start without it.

14. Database migrations

The schema is managed with EF Core code-first migrations and applied with `dotnet ef database update`. Migrations below are listed oldest to newest.

Migration	What it does
20261009012345_Initial1	Initial schema.
20261009012850_1	Early schema adjustment.
20261010214127_newMigration	Schema adjustment.
20261010221321_Migration for Creating Deleted wallet	Adds the deleted/closed wallet concept.
20261011100158_Ledger and Transactions	Introduces the ledger and transactions.
20261011144951_Ledger and Transaction	Ledger/transaction refinement.
20261012194855_Testing Idempotency	Adds idempotency records.
20261013014142_AddWalletRoleAndOrderId	Adds wallet role and transaction order id (merchant checkout).
20261013021851_SeedDb	Seeds the merchant account and wallet.
20261013023603_AddTransactionParent	Adds the parent transaction link (for refunds).
20261013132054_IdempotencyResponseText	Stores the full response body as text.
20261014001304_AddWalletAccountIdCreatedAtIndex	Adds the (account_id, created_at) index for stable listing.
20261014114752_AddAuth	Adds password support and the password-reset table.
20261014154432_MoneyScaleTo2	Locks money columns to two decimal places.
20261016230838_RemovePasswordReset	Drops the password-reset table (feature removed, see section 17).

> **Applying migrations:** use the Neon *direct* host (drop `-pooler`) for migrations — EF takes an advisory lock that misbehaves through a connection pooler. Keep the *pooled* string for the running application. There is no auto-migrate at startup; migrations are applied deliberately.

15. Testing

The test project runs real integration tests through `WebApplicationFactory`: each test spins up the API in-process and talks to the actual PostgreSQL database. There is roughly 46 tests across four files.

File	Focus
AuthTests.cs	Sign-up validation, duplicate email, login success and generic failures, the merchant account without a password, and that unauthenticated wallet access is rejected.
IntegrationTests.cs	The core money paths: create wallet, list and paginate, ledger consistency after fund and transfer, funding with idempotency (replay and failure marking), transfers (insufficient balance, closed wallets, same-wallet guard), checkout (merchant rule), refunds (partial, already-refunded, refund-wallet creation and reuse), amount range and decimal-place rules, and the merchant ledger balance check.
RateLimitTests.cs	Exceeding the login limit returns 429.
ExceptionContractTests.cs	Pins the status codes and messages that each exception maps to — a deliberate contract.

> **Important:** the tests share the same database as the running application (the connection string comes from user secrets). Point `ConnectionStrings__DefaultConnection` at a separate Neon branch before running the suite, so tests never touch production data. Tests clean up after themselves, so they are not idempotent if cancelled mid-run.

Test harness

A shared factory configures the web host for the Development environment and can dial the auth rate limit up (so ordinary tests are not throttled) or down (to a low limit for the rate-limit test). Test classes track the accounts they create and delete their wallets, ledger entries, transactions and accounts in `DisposeAsync`.

16. Deployment

The API is deployed as a Docker container on Render and the web app on Cloudflare Pages. The database is PostgreSQL hosted on Neon. Both are live:

Component	URL
API	https://apexpay-zqfr.onrender.com
Web app	https://apexpay-8ty.pages.dev

API container

A multi-stage Dockerfile builds the project with the .NET SDK image and runs it on the smaller ASP.NET runtime image. Render provides a `PORT` variable, which the container passes to the app as the listen URL:

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
... restore, publish ...

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
ENV ASPNETCORE_ENVIRONMENT=Production
CMD ["sh", "-c", "dotnet ApexPay.dll --urls http://0.0.0.0:${PORT:-10000}"]
```

Render environment variables

`ConnectionStrings__DefaultConnection`, `Jwt__Key`, `Frontend__Origin`, `AllowedHosts`. Secrets are set in the Render dashboard, never in the repository.

Cloudflare Pages

The web app is built with `npm run build` (TypeScript check, then Vite). Cloudflare Pages must be given `VITE_API_BASE_URL` at build time, pointing at the Render API URL. Without it the production bundle has no API address and the app will not start.

Database

PostgreSQL on Neon. The application uses the pooled connection string; migrations use the direct host. The seeder runs on first start and creates the merchant account and settlement wallet idempotently.

17. Known limitations and roadmap

Item	Detail
No password reset	The forgot/reset-password flow was removed. Transactional email requires an authenticated sender domain, and the free email tiers reject consumer senders such as `@gmail.com`. The password-reset table was dropped by the `RemovePasswordReset` migration. There is no self-service reset until a verified sending domain is available.
No health endpoint	A `/health` endpoint (checking database connectivity) is planned but not implemented.
No refresh tokens	Access tokens simply expire; the user logs in again. There is no refresh-token rotation.
Funding is simulated	Money is added directly; there is no real payment provider or bank transfer.
Single currency by default	Wallets default to NGN; there is no FX between currencies.
No API versioning	Routes are not versioned yet.

Possible next steps

- Add a `/health` endpoint backed by a database connectivity check.
- Restore password reset once a verified sending domain exists.
- Add refresh tokens for smoother sessions.
- Introduce a webhook or notification when a payment or refund completes.
- Add API versioning and an OpenAPI-generated client.

18. Appendix

Repository layout

```
ApexPay.slnx                  Solution (two projects)
Dockerfile                    API container image
.dockerignore
.gitignore
ApexPay/                      Web API project
  Program.cs                  Service registration, middleware, startup
  appsettings.json            Non-secret configuration
  Controllers/                AccountController, WalletController
  Service/                    WalletService, AccountService, IdempotencyService
  Service/Exceptions/         Exception hierarchy and status codes
  Dto/                        Request and response models
  Models/                     Entities and enums
  Database/                   ApexPayDbContext, DatabaseSeeder
  Migrations/                 EF Core migrations
  Properties/                 launchSettings.json
ApexPay.Test/                 xUnit integration tests
  ApexPayFactory.cs           Test web-host factory
  AuthTests.cs
  IntegrationTests.cs
  RateLimitTests.cs
  ExceptionContractTests.cs
ApexPay.Web/                  React web app
  index.html
  package.json
  vite.config.ts
  tsconfig.json
  .env.development / .env.production / .env.example
  src/
    main.tsx, App.tsx, styles.css, money.ts
    api/                      client.ts, types.ts
    auth/                     session.ts
    components/               ui.tsx, ProtectedRoute.tsx
    hooks/                    useWallets.ts
    pages/                    Login, Signup, Home, Wallets, WalletDetail, Fund, Transfer, Checkc
```

Glossary

Term	Meaning
Idempotency key	A client-supplied identifier that lets the same request be retried safely without repeating the effect.
Double-entry ledger	A bookkeeping model where every movement is recorded as a matching debit and credit, so totals always balance.
Debit / Credit	A ledger line that decreases (debit) or increases (credit) a wallet's balance in this system.
Merchant wallet	A wallet whose role is MERCHANT; the destination of checkout payments and the source of refunds.
Idempotency record	The stored result of a money-moving request (status code and body) keyed by path and key.
Stale key	An in-progress idempotency record left behind by a crash, which one caller may take over after a timeout.
Snake_case naming	EF Core naming convention that maps C# PascalCase to PostgreSQL snake_case automatically.
FOR UPDATE	A row lock taken during a refund so concurrent refunds of the same payment cannot race.
